

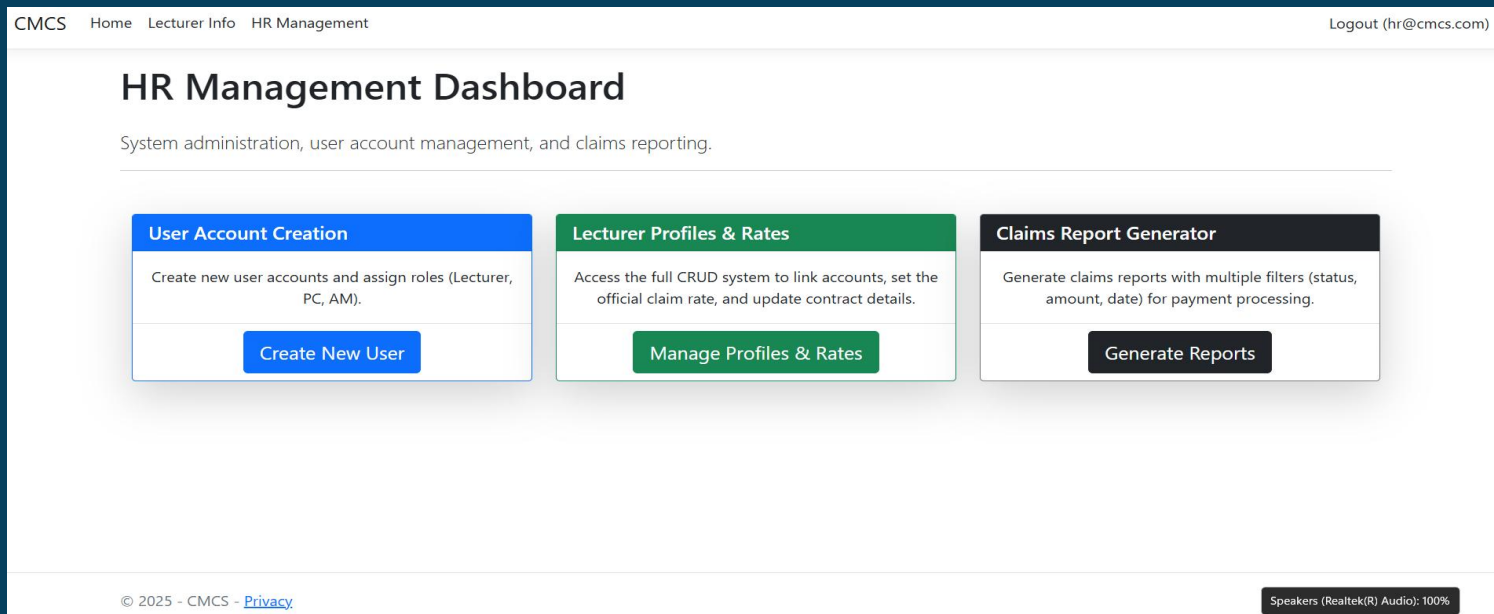


CMCS Presentation

ST10445696

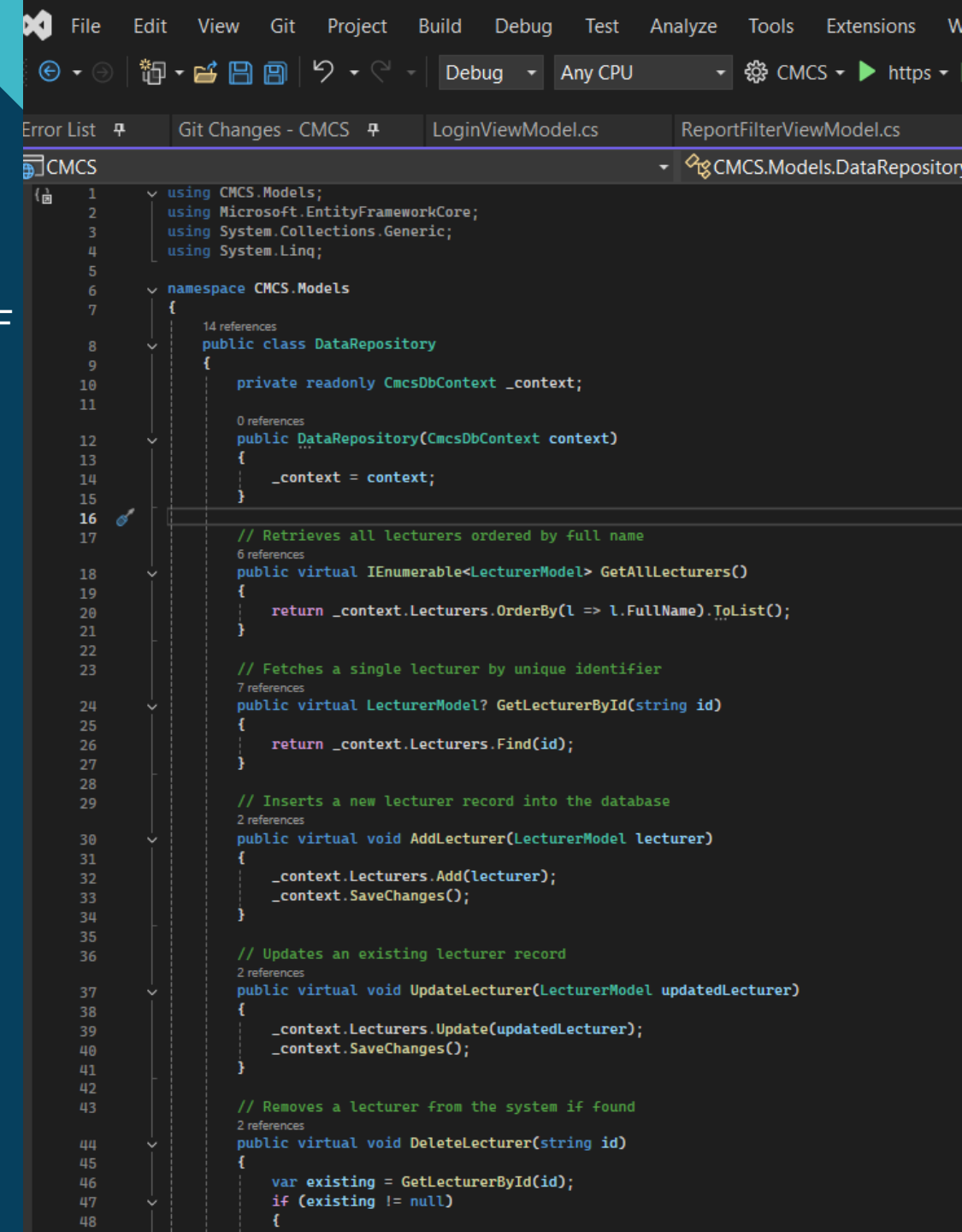
Explanation Of The System In My Own Words

- CMCS is a secure .NET mvc web app designed to automate the independent contractor claim and multi-stage approval workflow. It replaces manual paperwork with a secure, digital process for lecturers, program coordinator, academic manager, and HR.



Extra Features/Techniques Added.

- Database: I migrated from file storage to Entity Framework Core (EF Core) and SQL Server. The DataRepository now handles all communication with the database using efficient LINQ queries
- EF Technique: I used LINQ joins inside the LecturerInformationController to pull email data from the Identity tables and link it to the Lecturer's contract profile.



The screenshot shows the Visual Studio IDE with the CMCS.Models.DataRepository class open. The code is written in C# and includes the following elements:

- Using statements for CMCS.Models, Microsoft.EntityFrameworkCore, System.Collections.Generic, and System.Linq.
- A namespace declaration for CMCS.Models.
- A public class DataRepository with a private readonly CmcsDbContext _context.
- A constructor public DataRepository(CmcsDbContext context) that initializes _context = context.
- A comment // Retrieves all lecturers ordered by full name above the method GetAllLecturers().
- A method GetAllLecturers() that returns _context.Lecturers.OrderBy(l => l.FullName).ToList().
- A comment // Fetches a single lecturer by unique identifier above the method GetLecturerById().
- A method GetLecturerById() that returns _context.Lecturers.Find(id).
- A comment // Inserts a new lecturer record into the database above the method AddLecturer().
- A method AddLecturer() that adds a lecturer to _context.Lecturers and calls _context.SaveChanges().
- A comment // Updates an existing lecturer record above the method UpdateLecturer().
- A method UpdateLecturer() that updates a lecturer in _context.Lecturers and calls _context.SaveChanges().
- A comment // Removes a lecturer from the system if found above the method DeleteLecturer().
- A method DeleteLecturer() that checks if a lecturer exists and then removes it.

```
1 using CMCS.Models;
2 using Microsoft.EntityFrameworkCore;
3 using System.Collections.Generic;
4 using System.Linq;
5
6 namespace CMCS.Models
7 {
8     14 references
9     public class DataRepository
10     {
11         private readonly CmcsDbContext _context;
12
13         0 references
14         public DataRepository(CmcsDbContext context)
15         {
16             _context = context;
17         }
18
19         // Retrieves all lecturers ordered by full name
20         6 references
21         public virtual IEnumerable<LecturerModel> GetAllLecturers()
22         {
23             return _context.Lecturers.OrderBy(l => l.FullName).ToList();
24         }
25
26         // Fetches a single lecturer by unique identifier
27         7 references
28         public virtual LecturerModel? GetLecturerById(string id)
29         {
30             return _context.Lecturers.Find(id);
31         }
32
33         // Inserts a new lecturer record into the database
34         2 references
35         public virtual void AddLecturer(LecturerModel lecturer)
36         {
37             _context.Lecturers.Add(lecturer);
38             _context.SaveChanges();
39         }
40
41         // Updates an existing lecturer record
42         2 references
43         public virtual void UpdateLecturer(LecturerModel updatedLecturer)
44         {
45             _context.Lecturers.Update(updatedLecturer);
46             _context.SaveChanges();
47         }
48
49         // Removes a lecturer from the system if found
50         2 references
51         public virtual void DeleteLecturer(string id)
52         {
53             var existing = GetLecturerById(id);
54             if (existing != null)
55             {
56             }
```

Security And Role-Based Access Implementation.

- **RBAC** : Security is built on ASP.NET Identity. I defined four roles (HR, AM, PC, Lecturer) and used the [Authorize(Roles=RoleName)] attribute on every controller (ClaimStatusController, AcademicManagerController, etc.) to guarantee that users cannot access pages outside their assigned role, even by manipulating the URL.
- **Seeding**: The DbSeeder class creates all roles and the mandatory HR Admin user, establishing the system administrator.

```
using System.Threading.Tasks;

namespace CMCS.Controllers
{
    [Authorize(Roles = DbSeeder.Roles.ProgrammeCoordinator)]
    public class ProgrammeCoOrdinatorController : Controller
    {
        private readonly DataRepository _dataRepository;
        private const double MAX_HOURS_PER_MONTH = 150.0; // Define the policy

        // Constructor: Initializes the controller with the data repository de
    }
}
```

```
using CMCS.DataSeeding;
using Microsoft.AspNetCore.Identity;
using System.Threading.Tasks;

namespace CMCS.DataSeeding
{
    // Seeds initial roles and an HR admin user into the database
    public static class DbSeeder
    {
        // Defines the system roles
        public static class Roles
        {
            public const string HR = "HR";
            public const string AcademicManager = "AcademicManager";
            public const string ProgrammeCoordinator = "ProgrammeCoordinator";
            public const string Lecturer = "Lecturer";
        }

        // Default HR admin credentials
        private const string HR_EMAIL = "hr@cmcs.com";
        private const string HR_PASSWORD = "HRPassword123!";

        // Ensures all roles exist and creates the HR admin user if missing
        public static async Task SeedRolesAndAdminAsync(IServiceProvider serviceProvider)
        {
            var userManager = serviceProvider.GetRequiredService<UserManager<IdentityUser>>();
            var roleManager = serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();

            await EnsureRoleExistsAsync(roleManager, Roles.HR);
            await EnsureRoleExistsAsync(roleManager, Roles.AcademicManager);
            await EnsureRoleExistsAsync(roleManager, Roles.ProgrammeCoordinator);
            await EnsureRoleExistsAsync(roleManager, Roles.Lecturer);

            // Check if the HR user exists, create if not
            var hrUser = await userManager.FindByEmailAsync(HR_EMAIL);
        }
    }
}
```

High-Level Method Overview

A. **ClaimStatusController.Create(POST)**: This is the most important method. It automatically pulls the HR-set rate from the database and links the claim to the logged-in user's ID, completely ignoring the rate input by the lecturer (Automation Rule Enforcement).

ClaimStatusController.Create(POST)

```
CMCS.Controllers.ClaimStatusController ClaimStatusController(DataRepository dataRepository, FileUploadService fileUploadService)

// Create (POST): Submits a new claim, handles document upload, and saves the claim and document records.
[HttpPost]
[ValidateAntiForgeryToken]
2 references
public async Task<IActionResult> Create(ClaimInputViewModel model)
{
    if (ModelState.IsValid)
    {
        // Get logged-in user and lecturer profile (Rate Automation)
        var user = await _userManager.GetUserAsync(User);
        if (user == null) return Unauthorized();

        var lecturerProfile = _dataRepository.GetAllLecturers()
            .FirstOrDefault(l => l.IdentityUserId == user.Id);

        if (lecturerProfile == null)
        {
            TempData["ErrorMessage"] = "Error: Your lecturer profile is not linked or has not been set up by HR.";
            return RedirectToAction(nameof(Index));
        }

        // rate validation
        if (model.HourlyRate != lecturerProfile.ContractHourlyRate)
        {
            ModelState.AddModelError(nameof(model.HourlyRate),
                $"Automation Rule Violated: Claim rate must match the official rate (R{lecturerProfile.ContractHourlyRate:N2}) set by HR. Your rate input has been ignored.");

            return View(model);
        }

        var (success, message, documentModel) = await _fileUploadService.ProcessUploadAsync(model.SupportingDocument, model.ClaimId);

        if (!success)
        {
            ModelState.AddModelError("SupportingDocument", message);
            return View(model);
        }

        if (documentModel == null)
        {

```

ClaimStatusController.Create(POST)

```
if (documentModel == null)
{
    TempData["ErrorMessage"] = "A critical error occurred while creating the document record.";
    return RedirectToAction(nameof(Index));
}

// Use IdentityUserId and HR-set rate
var claim = new ClaimModel
{
    ClaimId = model.ClaimId,
    LecturerId = lecturerProfile.LecturerId, // CMCS Lecturer ID
    IdentityUserId = user.Id, // ASP.NET Identity User ID
    SubmissionDate = DateTime.Now,
    HoursWorked = model.HoursWorked,

    // Use the official HR-set rate
    HourlyRate = lecturerProfile.ContractHourlyRate,

    AdditionalNotes = model.AdditionalNotes,
    DocumentId = documentModel!.DocumentId,
    Status = ClaimStatus.AwaitingPCVerification
};

_dataRepository.AddDocument(documentModel!);
_dataRepository.AddClaim(claim);

TempData["SuccessMessage"] = $"Claim {claim.ClaimId} submitted successfully and awaiting Programme Co-ordinator verification.";
return RedirectToAction(nameof(Index));
}
return View(model);
```

0 references

High-Level Method Overview

- **B. ProgrammeCoOrdinatorController.Verify():**
This method performs the Automated Verification Rule by checking the 150-hour limit before allowing the claim to proceed to the next stage.

B. ProgrammeCoOrdinatorController.Verify():

```
// Verify (POST): Moves a claim from PC Verification to Awaiting AM Approval and records the action date.
//Automates claim verification against MAX_HOURS_PER_MONTH policy.
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public IActionResult Verify(string id)
{
    var claim = _dataRepository.GetClaimById(id);
    if (claim == null) return NotFound();

    // Business Rule Check Hours Policy

    if (claim.HoursWorked > MAX_HOURS_PER_MONTH)
    {
        // If the claim violates the policy, automatically reject it and stop verification
        TempData["ErrorMessage"] = $"AUTOMATED REJECTION: Claim {id} violates policy. Claimed hours ({claim.HoursWorked}h) exceed the limit of {MAX_HOURS_PER_MONTH} hours.";

        // Set to RejectedByPC status automatically
        claim.Status = ClaimStatus.RejectedByPC;
        claim.PcActionDate = DateTime.Now;
        _dataRepository.UpdateClaim(claim);

        return RedirectToAction(nameof(Index));
    }

    // If the policy check passes:
    claim.Status = ClaimStatus.AwaitingAMApproval; // Move to next stage
    claim.PcActionDate = DateTime.Now;
    _dataRepository.UpdateClaim(claim);

    TempData["SuccessMessage"] = $"Claim {id} verified and meets policy. Moved to Academic Manager Approval.";
    return RedirectToAction(nameof(Index));
}

// Reject (POST): Changes the status of a specific claim to 'RejectedByPC' and records the action date.
```

The background is a solid dark blue. A diagonal line runs from the bottom-left towards the top-right. To the left of this line is a lighter blue area. To the right of the line is a dark blue area. A thin, hatched pattern is visible along the diagonal line itself.

Thank You